GP2PL: From Generalized Planning Evidence to Certified BDI Plan Libraries for Temporally Extended Goals

Anonymous submission

Abstract

Generalized planning learns reusable behavior for problem families, whereas Belief–Desire–Intention (BDI) agents execute plan libraries. Direct adaptation leaves a representation gap: singleton-goal evidence may omit modules for subsidiary achievements and does not certify temporal composition. We introduce GP2PL (Generalized Planning to Plan Libraries), which compiles such evidence and Planning Domain Definition Language (PDDL) action models into certified BDI plan libraries. GP2PL validates and lifts evidence, generates schema-derived modules for producible goals, selects a compact closed atomic core, and appends query controllers guided by deterministic finite automata for finite-trace temporal goals without relearning that core. For the generated candidate language and supported fragment, accepted branches and controllers satisfy conditional soundness guarantees; uncertified obligations are rejected. Across 16 domains, five independently seeded cores achieve 98.68% mean held-out coverage (1.29 percentage-point sample standard deviation). Recursive candidates improve paired atomic success by 10.42 percentage points, and preservation certification improves paired temporal success by 9.28 percentage points. All 475 translated specifications match their gold finite-trace languages, and the fixed-core temporal study produces independently validated accepting traces for all 1,228 bound queries.

1 Introduction

Classical PDDL planning computes an action sequence for one grounded problem (McDermott et al. 1998). Generalized planning instead learns reusable behavior for a family of related problems (Srivastava et al. 2011; Chen et al. 2026). A Belief–Desire–Intention (BDI) interpreter uses a different representation: a plan library that connects events and belief contexts to actions and subsidiary goals (Rao 1996; Meneguzzi and De Silva 2015). Turning a generalized policy into such a library is therefore a compilation problem, not a change of syntax.

Blocks World is a standard planning domain in which a robot rearranges blocks. A lifted achievement `on(X,Y)` asks that block X rest on block Y. A learned rule for achieving it may use the action `stack(X,Y)` when the robot is holding X and the destination block is clear. If Y is not clear, however, an executable BDI library also needs a reusable plan for making it clear, even when that condition never appeared as a training goal. More generally, variables must remain safely bound, subsidiary goals must resolve to library plans, and recursive plans must make progress.

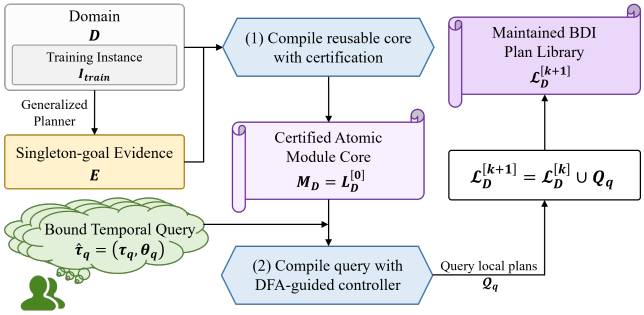

Figure 1: GP2PL separates reusable domain compilation from query-specific temporal compilation. From domain D, training instances $\mathcal{I}_{\text{train}}$, and raw singleton-goal evidence $\mathcal{E}_{\text{raw}}$, Validated policy lifting constructs the internally closed core $\mathcal{M}_D = \mathcal{L}_D^{[0]}$ once. DFA-guided temporal compilation maps a bound query $\hat{\tau}_q$ to query-local plans $\mathcal{Q}_q$ and updates the sole maintained library by $\mathcal{L}_D^{[k+1]} = \mathcal{L}_D^{[k]} \cup \mathcal{Q}_q$.

Temporally extended goals add a second representation gap (De Giacomo and Vardi 2013; De Giacomo and Rubin 2018). A request to place one block on another and only later clear a third block constrains a finite state trace, not only its final state. Likewise, a conjunctive milestone requires its conditions to hold together, so a later plan must not undo an earlier achievement. This is a form of BDI goal interference (Thangarajah, Padgham, and Winikoff 2003). Safe reuse therefore requires temporal control over the fixed domain library and observation after primitive actions.

The central design choice is reuse: temporal requests extend one maintained library instead of triggering a new domain-level learning run. Both compiler stages reason over typed action schemas and effects rather than domain names.

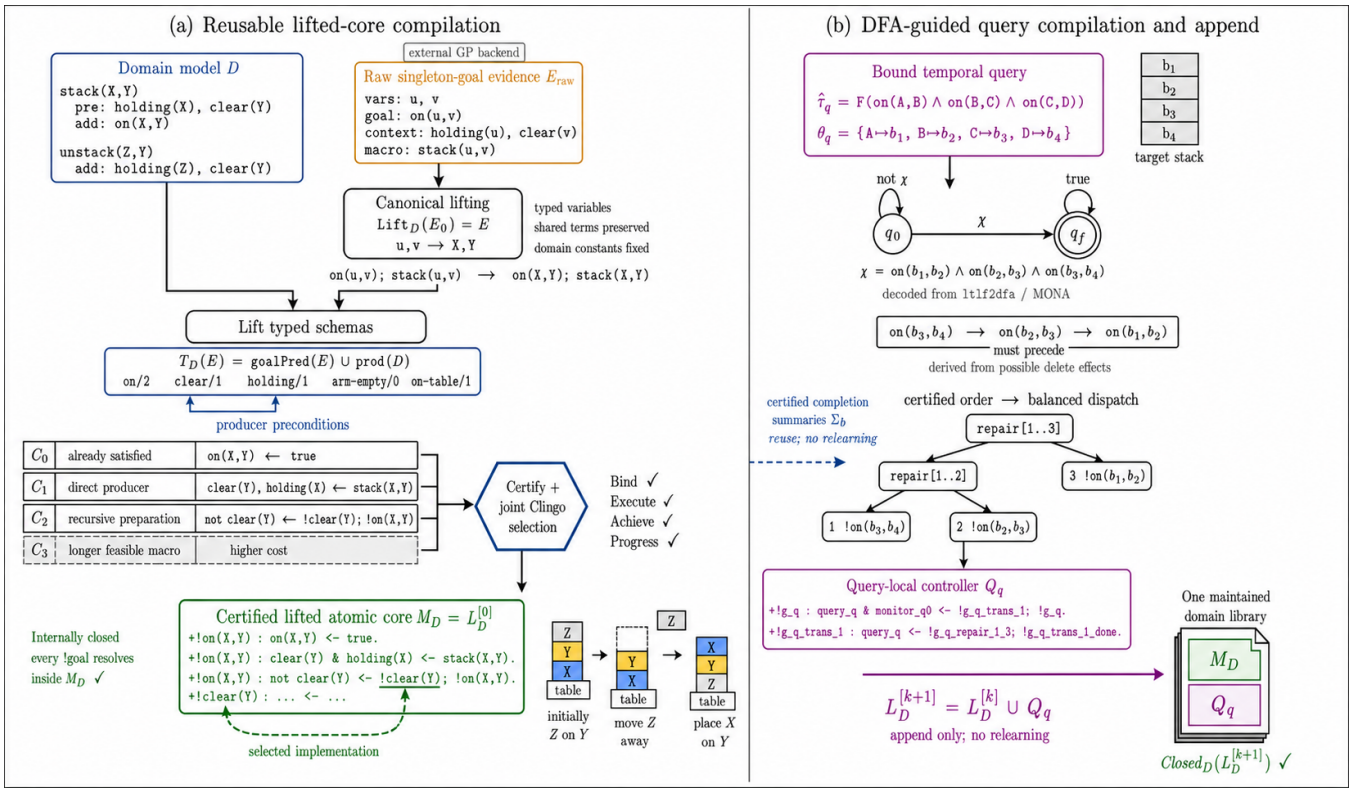

Figure 2: Certified lifting and temporal composition in Blocks World. Left: GP2PL canonically lifts singleton-goal evidence, augments it with typed action-schema candidates, and selects the internally closed atomic core $\mathcal{M}_D = \mathcal{L}_D^{[0]}$. Right: for a bound LTL_f query, it derives a preservation-safe order for each MONA-derived DFA progress guard, renders the balanced query-local controller $\mathcal{Q}_q$, and appends that controller without relearning the core.

GP2PL makes three contributions. First, it compiles singleton-goal evidence and PDDL schemas into an internally closed BDI core with modules for omitted producible subgoals.

Second, it accepts only branches with certificates for binding, symbolic execution, achievement, completion effects, and progress.

Third, it appends preservation-safe finite-trace controllers over the fixed core and monitors DFA progress after each primitive action.

The experiments separate component effects from complete-system behavior. Across 16 domains, independently compiled atomic cores average 98.68% held-out coverage over five evidence seeds. A fixed-core study validates all 1,228 registered temporal queries. These claims concern the declared candidate and formula fragments, not complete planning for arbitrary PDDL–LTL_f products. Formal definitions and proofs appear in Technical Supplement, Secs. 1–2; Sec. 5 gives the reporting and reproducibility protocol.

2 Problem Formulation and Foundations

2.1 Planning Domains and Generalized Evidence

We use typed STRIPS PDDL plus a bounded-integer resource fragment (Fikes and Nilsson 1971; McDermott et al. 1998; Fox and Long 2003). A domain is $D = \langle \mathcal{P}, \mathcal{F}, \mathcal{A}, \mathcal{T} \rangle$, where $\mathcal{P}$ and $\mathcal{F}$ contain predicate schemas and integer-valued resource functions, $\mathcal{A}$ contains action schemas, and $\mathcal{T}$ is a type hierarchy. A state $s = (s_{\mathcal{P}}, s_{\mathcal{F}})$ combines Boolean and integer valuations. For an applicable grounded action a, the Boolean successor is

$$\gamma_{\mathcal{P}}(s, a) = (s_{\mathcal{P}} \setminus del(a)) \cup add(a).$$

The numeric component admits finite integer values, comparisons with constants, and constant integer increases or decreases; arbitrary arithmetic and continuous change are excluded. A problem instance is $I = \langle D, O_I, s_I^0, G_I \rangle$, where O_I is its finite typed object set, s_I^0 its initial state, and G_I its PDDL achievement condition. A generalized-planning task supplies finite training and test instance sets that share D. The formal interface is agnostic to whether those finite sets are curated or produced by a parameterized problem generator; their provenance and separation remain part of the experimental protocol. MOOSE applies classical goal-regression ideas (Fikes, Hart, and Nilsson 1972; Alcázar et al. 2013) to singleton goal conditions and lifts the resulting plans into first-order condition-to-macro rules (Chen et al. 2026). We treat these rules as evidence: they demonstrate ways to achieve observed singleton goals, but need not constitute a closed or compact atomic module core.

2.2 BDI Plan Libraries and Their AgentSpeak(L) Realization

A procedural BDI plan rule has a triggering event, an applicability context, and a body (Meneguzzi and De Silva 2015). In the AgentSpeak(L) realization evaluated here (Rao 1996), an achievement plan is

$$+!p(\bar{X}) : C \leftarrow b_1; \ldots; b_m,$$

where C is a conjunction of beliefs and comparisons and each b_i is a PDDL action or an internal achievement call $!q(\bar{Y})$. A library is lifted when plans contain variables rather than training objects, and internally closed when every internal call resolves to a type-compatible selected module. Internal-call closure is a signature-resolution property; it does not assert that some plan for the call is applicable in every reachable state.

Our certificates apply to this trigger–context–body model and to the PDDL semantics of primitive actions. Formally, a candidate branch is $b = \langle g_b, C_b, \beta_b, \Sigma_b, \pi_b \rangle$, where g_b is one lifted achievement literal, C_b is a conjunctive applicability condition, β_b is a finite body of primitive actions and internal calls, Σ_b is a conservative conditional module-completion summary, and π_b records branch provenance. We map selected branches to AgentSpeak(L) and evaluate them with Jason (Bordini, Hübner, and Wooldridge 2007); claims about another BDI language would require a semantics-preserving translation and an independent execution study. Query controllers follow declarative goal patterns by rechecking their intended state after subsidiary plans return (Hübner, Bordini, and Wooldridge 2006).

2.3 Finite-Trace Temporal Goals

Linear temporal logic over finite traces (LTL_f) is interpreted over a nonempty finite state sequence (De Giacomo and Vardi 2013). For proposition alphabet AP_q, the declared input language Φ_{syn} is

$$\begin{aligned} \ell &::= a \mid \neg a, \\ \varphi &::= \ell \mid (\varphi \wedge \varphi) \mid F\varphi \mid X\varphi \\ &\quad \mid (\varphi U \varphi), \qquad a \in AP_q. \end{aligned}$$

Negation is restricted to atoms; the serialized syntax writes it as $!$. Both X and U have their strong finite-trace meanings. The evaluation family $\Phi_{\mathrm{bench}} \subset \Phi_{\mathrm{syn}}$ contains instances of the five predeclared profiles in Sec. 7. Membership in either set is not an action-strategy guarantee.

Automata-theoretic planning uses a finite automaton to track temporal progress (De Giacomo and Rubin 2018). We construct the deterministic finite automaton (DFA) $\mathcal{D}_q = \langle Q_q^{\mathrm{dfa}}, 2^{AP_q}, \delta_q, q_q^0, F_q \rangle$ with LTLf2DFA and MONA (Fuggitti 2020; Henriksen et al. 1995). Here Q_q^{dfa} is the finite state set, AP_q the proposition alphabet, δ_q the deterministic transition function, q_q^0 the initial state, and F_q the accepting set. The valuation map $val_q : \mathrm{States}(D) \to 2^{AP_q}$ interprets a PDDL state over that alphabet. The compiler-supported subset requires certified progress transitions. For automaton state z, let $d_q(z)$ be the shortest directed-path length to F_q, or ∞ when no accepting state is reachable. A progress transition (q_s, χ, q_t) satisfies $d_q(q_t) < d_q(q_s) < \infty$ and has a guard containing conjunction and literal negation. A supported guard χ has normal form

$$\chi = \bigwedge_{G \in P_\chi^+} G \wedge \bigwedge_{N \in P_\chi^-} \neg N, \qquad \mathcal{O}_\chi = \langle P_\chi^+, P_\chi^- \rangle,$$

where the signed transition obligation $\mathcal{O}_\chi$ separates positive achievements P_χ^+ from atoms P_χ^- that must remain absent. MONA may represent Boolean alternatives by several valuation cubes. GP2PL treats each cube as a separate conjunctive guard; several cubes are not one disjunctive guard, and the monitor continues to evaluate every original cube. An explicit disjunction inside one guard and strategy choice without a certificate remain unsupported. The certificate-dependent subset $\Phi_{\mathrm{cert}}(D, \mathcal{M}_D) \subseteq \Phi_{\mathrm{syn}}$ contains the validated bound formulas whose required DFA progress obligations admit the certificates below. The soundness claims apply to this subset; the empirical claims apply to accepted members of Φ_{bench}. A query-local monitor evaluates the DFA initially and after every successful primitive PDDL action. Arbitrary arithmetic and progress without a certified action strategy are outside this fragment.

2.4 Certified Compilation Problem

Compilation problem. Given D, training instances $\mathcal{I}_{\mathrm{train}}$, and normalized singleton-goal evidence E, atomic compilation produces the certified atomic module core $\mathcal{M}_D$: the query-independent branches whose triggers are singleton achievements.

An unbound temporal specification is τ_q, its externally supplied type-consistent object binding is θ_q, and $\widehat{\tau}_q = (\tau_q, \theta_q)$ is the validated bound query. Temporal compilation produces the query-local plan set $\mathcal{Q}_q$, comprising its top-level goal, DFA dispatch, and transition-repair plans. For appended queries $q_1, \ldots, q_k$, the one maintained domain library is

$$\mathcal{L}_D^{[0]} = \mathcal{M}_D, \qquad \mathcal{L}_D^{[k]} = \mathcal{M}_D \cup \bigcup_{i=1}^{k} \mathcal{Q}_{q_i}.$$

Thus $\mathcal{M}_D$ is the stable logical core of $\mathcal{L}_D^{[k]}$, not a second persisted library. Throughout, calligraphic symbols denote finite sets or structured artifacts; b denotes one candidate branch, e one normalized evidence rule, and χ one DFA transition guard. The Technical Supplement, Sec. 1 defines the generated candidate set, certificates, preservation portfolios, and transition-repair programs used below.

Positive predicate goals and supported integer equalities require an executable achieving module. Negative literals denote absence and preservation, never synthetic negative achievement calls. Conjunctive guards require a preservation-safe order; mixed Boolean–numeric guards require complete net effects for a common final state. Compilation rejects unsupported arithmetic, disjunction, cyclic threats without a preserving strategy, and any obligation whose effects are incomplete. In the AgentSpeak(L) realization, static sort-membership beliefs may constrain PDDL

type compatibility, but no domain-specific typing predicate becomes an achievement goal or primitive action. Formal definitions and supported-fragment assumptions appear in the Technical Supplement, Sec. 1.

3 Related Work and Positioning

Generalized planning. Generalized planning seeks compact policies or programs that solve families of planning instances (Jiménez, Segovia-Aguas, and Jonsson 2019; Srivastava et al. 2011; Chen et al. 2026). Feature approaches learn abstractions or policies from small examples (Bonet and Geffner 2018; Francès, Bonet, and Geffner 2021; Bonet and Geffner 2024). Policy sketches encode serialized subgoal structure, with answer-set optimization used to learn compact rules (Drexler, Seipp, and Geffner 2022, 2024); PG3 uses lifted decision lists (Yang et al. 2022), and BFGP++ uses terminating programs (Segovia-Aguas, E-Martín, and Jiménez 2022). MOOSE learns lifted condition-to-macro rules by singleton-goal regression (Chen et al. 2026). These methods learn reusable behavior. Recent BDI integrations instead invoke a classical planner at run time (Xu et al. 2024) or use generalized planning directly for means–ends reasoning (Meneguzzi, Fraga Pereira, and Oren 2025). GP2PL instead addresses the representation-compilation problem of transforming singleton-goal evidence, together with the domain's producible target universe and action-schema-derived candidates, into a query-independent atomic module core $\mathcal{M}_D$ that can be extended by later queries.

Modular policies and BDI plan libraries. Policy-reuse languages allow one generalized policy to invoke another with parameters (Bonet, Drexler, and Geffner 2024), motivating internal calls such as `!clear(Y)`. Procedural BDI languages likewise organize behavior as event-triggered plans selected from a context-dependent library (Meneguzzi and De Silva 2015; De Silva, Meneguzzi, and Logan 2020); AgentSpeak(L) formalizes these plans and Jason executes an extension (Rao 1996; Bordini, Hübner, and Wooldridge 2007). Plan patterns for declarative goals recheck intended states after procedural return (Hübner, Bordini, and Wooldridge 2006); plan-failure work separates a failed attempt from continued goal pursuit (Sardina and Padgham 2007). Definite and potential effect summaries detect interference between BDI goals (Thangarajah, Padgham, and Winikoff 2003). GP2PL adopts these execution principles and derives certified modules, conditional module-completion summaries, and declarative completion controllers from planning evidence and PDDL schemas.

Finite-trace temporal control. LTL_f provides finite-trace semantics and admits automata-based reasoning (De Giacomo and Vardi 2013). We instantiate the automata-theoretic translation with the MONA-backed LTLf2DFA construction (Fuggitti 2020; Henriksen et al. 1995). Full temporal synthesis synthesizes over the domain–automaton product (De Giacomo and Rubin 2018). GP2PL instead composes certified repairs over an existing BDI library and makes no complete product-synthesis claim. Temporally extended goals also have explicit agent-language semantics (Hindriks, van der Hoek, and van Riemsdijk 2009); we retain standard plan execution and add a query-local monitor. Plan4Past motivates holding the underlying planner fixed in a temporal comparison (Bonassi et al. 2023). FOND4LTLf (De Giacomo and Fuggitti 2021) is therefore a task-level reference on its accepted subset, not a compiler ablation.

Natural-language temporal specifications. Prior systems map natural-language instructions to temporal logic through extensible pattern libraries (Fuggitti and Chakraborti 2023), lifted language-to-logic translation (Chen et al. 2023), or modular grounding in new environments (Liu et al. 2023); related planning work extracts ordering and blocking constraints into solver specifications (Guo, Kingston, and Kavraki 2025). Prompt programs can constrain structured outputs (Beurer-Kellner, Fischer, and Vechev 2023), but prompt-based semantic parsers remain sensitive to input perturbations (Zhuo et al. 2023). Our evaluated translation protocol therefore constrains the schema and vocabulary, then validates typed, externally bound parametric LTL_f by exact finite-trace language equivalence before GP2PL compilation. This protocol tests controlled structured translation, not robustness to unrestricted natural language. The complete prompt appears in the Technical Supplement, Sec. 4.2.

Position of this work. Generalized planners target policies or programs, BDI programming assumes a plan library, and temporal synthesis targets an automaton strategy. GP2PL connects them through certificate-carrying compilation. The broader proof-carrying principle attaches machine-checkable evidence to executable artifacts (Necula 1997); GP2PL's certificates are narrower symbolic planning obligations. GP2PL contributes the certified construction of a reusable atomic module core and preservation-safe query plans in the maintained BDI library $\mathcal{L}_D^{[k]}$. Goal regression, answer-set solving, AgentSpeak semantics, and LTL_f-to-DFA translation remain prior components.

4 Certified Atomic-Module Compilation

4.1 Method Overview

GP2PL first maps $D, \mathcal{I}_{\mathrm{train}}, \mathcal{E}_{\mathrm{raw}} \longrightarrow \mathcal{M}_D$, compiling the query-independent certified atomic module core from a PDDL domain, training split, and generalized-planning evidence. Temporal queries do not enter this optimization; every later query reuses $\mathcal{M}_D$. Algorithm 1 summarizes the procedure.

Certification and selection operate on trigger–context–body rules before surface rendering. AgentSpeak(L) rendering preserves the selected triggers, contexts, bodies, and internal-call relation in $\mathcal{M}_D$. These selected rules define the compiler output; AgentSpeak(L) and Jason provide its evaluated realization. Table 1 states the acceptance and rejection boundary used by the formal claims and experiments.

4.2 Evidence Normalization and Canonical Lifting

An evidence provider is admissible when its raw artifact $\mathcal{E}_{\mathrm{raw}}$ can be parsed into a provider-normalized singleton-goal evidence program E_0. Canonical lifting then maps E_0

Algorithm 1 Compile the Certified Lifted Atomic Module Core

Require: PDDL domain D, training split $\mathcal{I}_{\text{train}}$, raw provider artifact $\mathcal{E}_{\text{raw}}$
Ensure: Certified lifted atomic module core $\mathcal{M}_D$ or a rejection diagnostic

$E_0 \leftarrow \textsc{NormalizeEvidence}(\mathcal{E}_{\text{raw}}, D, \mathcal{I}_{\text{train}})$
$E \leftarrow \textsc{CanonicalLift}(E_0, D)$
$T_D(E) \leftarrow \textsc{ProducibleTargets}(E, D)$
$\mathcal{C}_E \leftarrow \textsc{ValidateEvidenceMacros}(E, D)$
$\mathcal{C}_D(E) \leftarrow \textsc{LiftSchemas}(D, \mathfrak{G}, T_D(E), T_D^{\mathbb{Z}}(E))$
$\mathcal{C}_{D,E}^{\checkmark} \leftarrow \textsc{Certify}(\mathcal{C}_E \cup \mathcal{C}_D(E), D)$
$\mathfrak{F}_{D,E} \leftarrow \textsc{FeasibleSubsets}(\mathcal{C}_{D,E}^{\checkmark}, E)$
if $\mathfrak{F}_{D,E} = \varnothing$ **then**
  reject
end if
return $\mathcal{M}_D \leftarrow \textsc{ClingoLexMin}(\mathfrak{F}_{D,E})$

Construct	Accepted strategy	Rejection boundary
Positive predicate	Certified atomic module	No executable achieving branch
Numeric equality	Bounded constant-integer progress	Unsupported arithmetic or incomplete numeric effect
Positive conjunction	Threat-safe serialization	Incomplete effects or cyclic threats
Negative predicate	Observe absence or use an exact certified deleter	Synthetic negated achievement or possible re-addition
Mixed Boolean and numeric guard	Complete net-effect and preservation certificate	Incomplete mixed-effect certificate
Boolean alternatives	Separate conjunctive MONA cubes	Explicit disjunction in the input or one cube; uncertified strategy choice
Primitive-state safety	DFA monitor after every primitive action	Completion-only observation

Table 1: Supported compilation fragment and fail-closed boundary. Rejection means that GP2PL emits a structured diagnostic rather than an uncertified controller or synthetic atomic goal.

to $E = \mathrm{Lift}_D(E_0)$, whose rules have the form

$$e = \langle \bar{X}_e, C_e^s, g_e, \bar{a}_e, \nu_e, \pi_e \rangle,$$

where C_e^s is a partial state condition, g_e is one goal atom, $\bar{a}_e$ is a PDDL action macro, ν_e records numeric conditions and effects, and π_e records provenance. For example, evidence for `at(P,L)` may contain a `load; move; unload` macro. Admissibility requires every symbol and arity to occur in D, capture-avoiding renaming of local variables, and a singleton positive achievement target. The lifting map alpha-normalizes provider-local variables into typed canonical variables, preserves repeated-term sharing, and leaves constants declared by D fixed. It does not infer a first-order policy from ground plans: that operation belongs to the external evidence provider. For example, a provider rule whose declared variables occur as `on(block0,block1)` and `stack(block0,block1)` becomes `on(X,Y)` and `stack(X,Y)`, while a PDDL domain constant remains unchanged. In the experiments, E_0 is obtained from MOOSE readable policies (Chen et al. 2026). Negative achievement evidence is rejected because those policies provide no validated action realization for achieving a negated literal.

4.3 Action-Schema Candidate Generation

Let $\mathrm{goalPred}(E)$ contain the predicate symbols targeted by positive predicate evidence, and let

$$\mathrm{prod}(D) = \{p \mid \exists a \in \mathcal{A},\ \exists \ell \in \mathrm{add}(a) : \mathrm{pred}(\ell) = p\}.$$

The domain-wide producible target universe is

$$T_D(E) = \mathrm{goalPred}(E) \cup \mathrm{prod}(D).$$

Thus every predicate with a positive producer receives candidate atomic modules even when the evidence provider never observed it as a goal. A static predicate appears in no add or delete effect and remains context-only. A dynamic predicate with no positive producer is not invented as a positive achievement target unless admissible evidence explicitly supplies a validated realization. Supported numeric equalities form $T_D^{\mathbb{Z}}(E)$, the set of admitted singleton integer-equality targets in E; numeric-only evidence does not suppress $\mathrm{prod}(D)$.

The finite candidate-construction grammar is

$$\mathfrak{G} = \{\text{producer}, \text{regression}, \text{recursion}, \text{precondition repair}, \text{resource discharge}, \text{numeric progress}\}.$$

For the Boolean and numeric targets, LiftSchemas computes

$$\mathcal{C}_D(E) = \mathrm{Inst}_D(\mathfrak{G}, T_D(E), T_D^{\mathbb{Z}}(E)).$$

It instantiates these constructors using only typed schemas and effects from D and alpha-normalizes every resulting variable-level branch. Its output is finite because D and the target signatures are finite, constructor states are identified up to alpha-normalization, and no requirement/producer state or resource mode may repeat. For a Boolean target predicate p, a producer is an action schema $a \in \mathcal{A}$ for which some $\ell \in \mathrm{add}(a)$ satisfies $\mathrm{pred}(\ell) = p$. For target set T, define the reachable producer–precondition graph

$\mathcal{G}_D^{\mathrm{prod}}(T) = (V_T, E_T)$ over predicate signatures: an edge $p \to r$ belongs to E_T exactly when a producer for p has a positive dynamic precondition with predicate r, and V_T contains the signatures reachable from T through these edges. Backward STRIPS regression (Fikes, Hart, and Nilsson 1972; Alcázar et al. 2013; Chen et al. 2026) replaces one open positive precondition by a producer's preconditions and rejects a step whose deletes contradict another open requirement. Producer preconditions are first unified with compatible open requirements, after which only still-unbound parameters receive fresh variables. Search is enabled only when $\mathcal{G}_D^{\mathrm{prod}}(T)$ is acyclic.

Termination does not use an action-depth constant: a path cannot repeat an alpha-normalized requirement/producer step or resource mode, and only the shortest body is retained for an equivalent open-requirement and completion-effect summary. Every candidate is then verified by symbolic progression. Evidence macros are not truncated. Let $\mathcal{C}_E$ denote validated evidence branches and $\mathcal{C}_D(E)$ the branches generated from D for these target sets. Cyclic dependencies require provider evidence or a separately certified recursive module. The resulting candidate set $\mathcal{C}_{D,E} = \mathcal{C}_E \cup \mathcal{C}_D(E)$ is finite, but $\mathfrak{G}$ is not a complete hypothesis language for PDDL planning. Types constrain unification; an AgentSpeak(L) realization may expose static sort membership in contexts, but never as an achievement call or exported action.

4.4 Candidate Soundness Certificates

Every candidate carries a machine-checkable certificate. Its conditional module-completion summary Σ_b has Boolean projections $\mathrm{MustAdd}_b$, MayAdd_b, and $\mathrm{MayDelete}_b$, together with any exact constant-integer delta and keyed resource-mode transition. Table 2 connects each branch constructor to its additional proof obligation and the failure it excludes.

Binding, symbolic execution, and internal-call closure. Action and subgoal variables must be bound by the trigger, positive context, or certified static sort membership; negative contexts and comparisons only filter. Symbolic progression proves each precondition and final trigger achievement, recording $\mathrm{MustAdd}$, MayAdd, and $\mathrm{MayDelete}$. For any BDI plan set $\mathcal{R}$, define

$$\begin{aligned}\mathrm{Closed}_D(\mathcal{R}) \iff \forall b \in \mathcal{R},\ \forall! q(\bar{Y}) \text{ occurring in } \beta_b,\\ \exists b' \in \mathcal{R} \text{ such that}\\ g_{b'} \text{ type-unifies with } q(\bar{Y})\\ \text{under the type hierarchy of } D.\end{aligned}$$

The compiler requires $\mathrm{Closed}_D(\mathcal{M}_D)$, so every internal achievement call resolves to a type-compatible selected plan, including calls reached transitively. This set-level property is a Clingo obligation; it is distinct from constructing $T_D(E)$. It proves resolution by signature and type, not applicability of some branch in every reachable state.

Well-founded recursive progress. A same-predicate recursive branch b requires a ranking function $\rho_b : \mathrm{States}(D) \to \mathbb{N}$ defined by a non-negative relation count or anchored acyclic-cone count that strictly decreases and has an already-satisfied, direct-producer, or validated-evidence terminal branch. Threats are checked over the preparation graph selected by Clingo, not over unselected candidates. A branch may add the ranked relation outside the protected cone only when its trigger preserves the same anchor and a guard proves that the new anchor differs from the protected one. The ranking feature is used only in certification and never becomes an agent belief. The construction follows relational progress witnesses in general-policy work (Francès, Bonet, and Geffner 2021), but claims only schema-certified features from the generated candidate set.

Target-preserving resource discharge. Effects, not predicate names, identify temporary occupation of a reusable resource. Suppose an acquisition action deletes an availability fact $\mathit{free}(R)$ and adds an occupancy relation $\mathit{held}(R, U)$. These names are illustrative: the compiler infers the shared resource argument R and occupant argument U from the effect structure and obtains the same result after consistent symbol renaming. The resulting finite abstract mode graph is $\mathcal{G}_b^{\mathrm{res}} = (V_b^{\mathrm{res}}, E_b^{\mathrm{res}})$: vertices are alpha-normalized keyed occupancy modes, and an edge labelled by action schema a records a symbolically executable transition between two modes while preserving the target. A discharge body is accepted only if symbolic execution removes the current occupancy mode, restores the corresponding availability fact, leaves the atomic target true, and does not recreate an earlier normalized mode. Excluding repeated modes makes the discharge search finite without assuming that release is one action. Disequalities arise only when unifying two roles would make a PDDL precondition false.

Ranked cross-module precondition repair. Preparing one of several dynamic preconditions may temporarily change another, such as an object's location. A repair branch may defer an unrelated sibling only through a schema-inferred single-valued fluent when every deferred sibling has one producer, all static feasibility witnesses remain, local variables are alpha-renamed, and the original target is retried before primitive production. Cross-predicate repair candidates are optional. For a candidate core $\mathcal{M}$, Clingo assigns each trigger signature in its selected call graph a dependency rank $\kappa_{\mathcal{M}}$ and requires $\kappa_{\mathcal{M}}(g) > \kappa_{\mathcal{M}}(h)$ for every selected call edge $g \to h$. Same-predicate recursion instead requires the relational certificate above.

4.5 Feasible-Core Optimization

Validated evidence macros and certified action-schema-derived candidates enter one answer-set optimization problem solved with Clingo (Gebser et al. 2019). Evidence rules induce coverage obligations. A selected candidate may cover an evidence obligation only through alpha-equivalent triggers and bodies, a weaker conjunctive context with the same body, or an action-schema producer refinement with the same $\mathrm{MustAdd}$, no additional $\mathrm{MayDelete}$, and equivalent numeric and keyed-resource summaries. We write this evidence-coverage relation as $\mathrm{covers}_D(b, e)$. Syntactic body-prefix similarity is not treated as semantic equivalence.

Branch constructor	Additional acceptance obligation	Excluded failure
Validated evidence macro	every macro step maps to a PDDL action schema and symbolic execution establishes the singleton evidence target	invalid or misbound provider evidence
Direct producer	one action adds the target under the certified context and preserves its completion summary	action chosen only by predicate name
Acyclic regression	each open positive requirement is discharged without a repeated normalized producer role or conflicting delete effect	arbitrary depth cutoffs or cyclic schema search
Relational recursion	a non-negative relation feature strictly decreases and an already-satisfied, direct-producer, or validated-evidence terminal branch remains available	recursive oscillation
Target-preserving resource discharge	a finite non-repeating mode path restores keyed availability while preserving the target	unreleased reusable resource
Ranked precondition repair	Clingo selects an acyclic call graph with strictly decreasing natural-number dependency ranks	mutually recursive preparation without a progress proof

Table 2: Candidate constructors and soundness obligations for atomic BDI modules. Every selected branch additionally satisfies typed binding, symbolic PDDL executability, and target achievement on return; the resulting core satisfies $\mathrm{Closed}_D(\mathcal{M}_D)$.

The solver also treats every nonrecursive action-schema-derived branch as a schema-achievement obligation: at least one selected branch must realize the same certified achievement contract. Let $\mathcal{C}_D^{\mathrm{nr}}(E)$ denote these nonrecursive branches and let $\mathrm{realizes}_D(b, c)$ hold when selected branch b is equivalent to, soundly refines, or gives a certified target-preserving resource-discharge alternative to schema branch c. It further enforces internal-call closure, acyclic ranked cross-predicate preparation, and compatibility of selected effects with recursive rankings. Define

$$\mathcal{C}_{D,E}^{\checkmark} = \{b \in \mathcal{C}_{D,E} \mid \mathrm{Cert}_D(b)\},$$
$$\mathfrak{F}_{D,E} = \{\mathcal{M} \subseteq \mathcal{C}_{D,E}^{\checkmark} \mid \mathcal{M} \models \Omega_{D,E} \wedge \mathrm{Closed}_D(\mathcal{M})\}.$$

where $\Omega_{D,E}$ contains evidence coverage, schema-achievement coverage, preparation acyclicity, and recursive-ranking compatibility. Resource discharge is already part of the per-branch predicate Cert_D. In particular, schema-achievement coverage requires

$$\forall c \in \mathcal{C}_D^{\mathrm{nr}}(E), \quad \exists b \in \mathcal{M} : \mathrm{realizes}_D(b, c).$$

The optimizer uses the lexicographic objective

$$\mathcal{M}_D = \operatorname*{lexargmin}_{\mathcal{M} \in \mathfrak{F}_{D,E}} \big\langle -N_{\mathrm{rel}}(\mathcal{M}), -N_{\mathrm{prep}}(\mathcal{M}), |\mathcal{M}|, N_C(\mathcal{M}), N_\beta(\mathcal{M}) \big\rangle,$$

where N_{rel} and N_{prep} count compatible relation-ranked recursion and acyclic precondition-repair capabilities, N_C counts context literals, and N_β counts body steps. Clingo computes this core directly for the grounded finite instance. The selected trigger–context–body rules, rather than their AgentSpeak rendering, define $\mathcal{M}_D$. The claim is global only within $\mathcal{C}_{D,E}^{\checkmark}$, not over ungenerated programs.

The compiler rejects an untyped predicate with several producers when their safe use requires different nested precondition-repair branch sets. For example, if one `at/2` producer preserves a ranked relation and another may increase it, static role names alone do not prove that their object sets are disjoint. The compiler neither recognizes those names nor assumes disjointness.

5 DFA-Guided Composition of Temporally Extended Goals

Validated parametric input. GP2PL does not prescribe a controlled surface grammar for user requests. In the evaluated front end, a source utterance is accompanied by the public PDDL symbol catalogue, declared typed parameters, and binding constraints. Translation produces a structured specification $\tau_q = \langle \iota_q, \varphi_q, \mu_q, \Theta_q, \Gamma_q \rangle$, where ι_q is its identifier, φ_q is an LTL_f formula over proposition symbols, μ_q maps each proposition to a lifted PDDL predicate or integer equality, Θ_q is the typed parameter signature, and Γ_q contains binding constraints. Before automaton construction, validation requires a bijective formula-to-atom mapping, membership in the domain vocabulary, and correct arities, types, numeric values, temporal operators, and literal-only negation. Invalid specifications are rejected rather than repaired. The controlled utterances in the released benchmark define the evaluation distribution, not a required user-facing grammar; the serialized schema and translation protocol appear in the Technical Supplement, Sec. 4.2.

A runtime binding is a type-consistent map $\theta_q : \bar{X}_q \to O_I$ satisfying the type signature $\Theta_q : \bar{X}_q \to \mathcal{T}$ and constraints Γ_q; the validated bound query is $\widehat{\tau}_q = (\tau_q, \theta_q)$, and g_q denotes its generated top-level BDI goal. Applying θ_q substitutes declared parameters and leaves PDDL domain constants fixed. For $a \in AP_q$, its bound PDDL-state valuation is

$$a \in val_q(s) \iff \begin{cases} p(\bar{t})\theta_q \in s_{\mathcal{P}}, & \mu_q(a) = p(\bar{t}), \\ s_{\mathcal{F}}(f(\bar{t})\theta_q) = c, & \mu_q(a) = [f(\bar{t}) = c]. \end{cases}$$

Thus predicate propositions denote grounded Boolean facts and numeric propositions denote exact bounded-integer equalities in the same observed state. As defined in Sec. 2, LTLf2DFA/MONA produces

$$\mathcal{D}_q = \langle Q_q^{\mathrm{dfa}}, 2^{AP_q}, \delta_q, q_q^0, F_q \rangle$$

and its guarded transitions (Fuggitti 2020; Henriksen et al. 1995). GP2PL attaches a controller to each progress edge

(q_s, χ, q_t) with $d_q(q_t) < d_q(q_s) < \infty$; an accepting `true` self-loop needs none.

Same-source/same-target valuation cubes may share a common achievement objective, but the monitor still evaluates every complete cube. They remain separate conjunctive alternatives in MONA's transition relation, not one admitted disjunctive guard.

Each execution applies θ_q: $on(X, Y)$ under $X{=}b_1, Y{=}b_4$ calls $!on(b_1, b_4)$ without grounding the shared $+!on(X, Y)$ module. A binding inconsistent with the validated query specification is rejected. For guard $\chi = \bigwedge_{G \in P_\chi^+} G \wedge \bigwedge_{N \in P_\chi^-} \neg N$, the signed transition obligation is $\mathcal{O}_\chi = \langle P_\chi^+, P_\chi^- \rangle$. Compilation computes conditional module-completion summaries and a threat-induced precedence graph, building on BDI goal-interference analyses that distinguish definite from potential plan effects (Thangarajah, Padgham, and Winikoff 2003). An acyclic graph is topologically ordered; a cyclic graph requires either an occurrence-specific preservation portfolio or a joint guard-establishment branch b_χ^{joint} whose complete net effect establishes the entire guard. The transition is rejected if neither can be certified. When b_χ^{joint} establishes the complete guard, it is available from every positive occurrence it establishes and retains every query variable appearing in $\mathcal{O}_\chi$. Thus an already-true first literal cannot hide the branch from an unmet sibling or discard a query binding. The certified serialization $\boldsymbol{\ell}_\chi$ is rendered as a balanced binary transition-repair tree by Algorithm 2.

5.1 Conditional Module-Completion Summaries

A relational fixed point over selected calls computes Σ_b, the effects observable when branch b returns. Query arguments remain anchors; local variables are alpha-normalized and typed; effects retain branch conditions. Sequential composition records net polarity, so an internally deleted-and-restored atom is not a completion delete.

For occurrence $G_i \in P_\chi^+$, a preservation portfolio $\Pi_{\chi,i} \subseteq \mathcal{M}_D$ is the set of certified branches permitted to establish G_i under the obligations protected at that position. The threat-induced precedence relation is defined by $G_j \prec_\chi G_i$ exactly when a feasible $\mathrm{MayDelete}$ effect of a branch in $\Pi_{\chi,j}$ unifies with G_i. The graph $\mathcal{G}_\chi = (P_\chi^+, \prec_\chi)$ therefore requires G_j to be repaired before G_i. Single-valued invariants reject conjunctions requesting two values for one key.

A negative guard $\neg N$ never becomes $!\neg N$. Positive repairs must exclude feasible $\mathrm{MayAdd}(N)$. If N holds, a signed leaf may call only a finite branch with exact $\mathrm{MustDelete}(N)$ that establishes a positive sibling and preserves all obligations. A negative-only edge observes absence. Direct- deleter parameters bind through compatible sibling add effects before range restriction, without recognizing fluent names.

5.2 Preservation Portfolios for Cyclic Interference

For a cycle, each $\Pi_{\chi,i}$ retains only branches whose exact completion preserves prior positives and negatives. In the AgentSpeak realization, a query-local trigger exposes exactly the bodies rendered from that branch set. Portfolios are certified per ordered literal occurrence: two occurrences of p may need different portfolios when their protected prefixes differ, while identical portfolios may share one restricted module in the AgentSpeak realization. Contexts remain alternative applicability cases, not simultaneous obligations. A recursive $!p; !g$ branch also requires a complete preserving summary for p and redirects $!g$ to the occurrence's restricted module. Certificates record source branches by literal index and claim recursive preservation coverage only when at least one branch was selected. For a binary support relation R, the same-predicate rank ρ_b may be instantiated as support depth $\rho_R(x)$ only when its grounded request graph is functional and acyclic in every reachable state. Every selected branch must still preserve earlier achievements; otherwise compilation rejects.

5.3 Numeric and Joint Guard-Establishment Certificates

A positive integer equality may call a complete constant-delta action: unit deltas require strict direction, larger deltas the exact predecessor. Mixed guards use complete action-only net effects. For example, a body for $at(P, L) \wedge fuel(V) = 3$ must symbolically establish $at(P, L)$ and the exact integer value 3 in its final state while preserving every other signed guard obligation. A cyclic guard admits one joint guard-establishment action only if the same proof covers the complete successor valuation required by $\mathcal{O}_\chi$. Negated equality is observation-only without a certified change-away action; arbitrary arithmetic and uncertified sequential composition are outside the supported fragment.

Numeric preparation must strictly reduce the lexicographic ranking $\rho_{\text{num}} = \langle n_{\text{miss}}, d_{\text{target}} \rangle$, where n_{miss} is the number of missing producer preconditions and $d_{\text{target}}(s) = |f(s) - c|$ is the bounded deficit for target equality $f = c$. It must preserve other conditions and leave the target fluent unchanged until the establishing step. Compilation rejects the candidate when no decreasing tuple can be certified.

For strong Until, let $\mathcal{W}_{q_s}$ be the set of non-progress self-loop guards at source q_s. Their signed source invariants are

$$\mathcal{I}_{q_s} = \langle I_{q_s}^+, I_{q_s}^- \rangle, \quad I_{q_s}^\pm = \bigcap_{\chi \in \mathcal{W}_{q_s}} P_\chi^\pm.$$

Before the single progress literal, no action prefix may delete a positive invariant or add a negative one; the establishing step may consume the left operand. Repeated unit progress separates action and protected objects by a derived non-unification guard, and unavoidable consumption requires the exact predecessor. An observed target equality is the base case.

Inserted auxiliary variables are alpha-renamed away from query and producer variables, preventing textual collisions from imposing unrelated constraints.

5.4 Balanced Binary Transition-Repair Trees

Let $\boldsymbol{\ell}_\chi = (\ell_1, \ldots, \ell_n)$ be a certified serialization of $\mathcal{O}_\chi$ and let $\Pi_{\chi,i}$ be the optional preservation portfolio for occurrence ℓ_i. A positive leaf observes or achieves its atom; a negative leaf observes absence or invokes its exact certified deleter. The tree indexes these occurrences, not DFA states or primitive actions. For query q, let $\mathcal{R}_{q_s,\chi}$ denote the plans for one

source-state transition obligation and let the query-local plan set $\mathcal{Q}_q$ contain the top-level dispatcher together with every accepted $\mathcal{R}_{q_s,\chi}$. Algorithm 2 constructs one such transition plan set.

Algorithm 2 Compile a Certified DFA Progress Controller

Require: DFA source q_s, guard χ, signed obligation $\mathcal{O}_\chi$, selected atomic core $\mathcal{M}_D$
Ensure: Transition-repair plan set $\mathcal{R}_{q_s,\chi}$ or rejection
$\mathbf{\Sigma} \leftarrow \{\Sigma_b \mid b \in \mathcal{M}_D\}$
$(\boldsymbol{\ell}_\chi, \mathbf{\Pi}_\chi) \leftarrow \textsc{CertifySerialization}(\mathcal{O}_\chi, \mathbf{\Sigma})$
if $(\boldsymbol{\ell}_\chi, \mathbf{\Pi}_\chi) = \bot$ **then**
  reject
end if
emit entry $r_{q_s,\chi} \leftarrow !R_\chi[1,n]; !c_{q_s,\chi}$
procedure $\textsc{Build}(i,j)$
if $i = j$ **then**
  emit the satisfied plan for signed literal ℓ_i
  if $\Pi_{\chi,i} \neq \varnothing$ **then**
    emit the unmet plan restricted to $\Pi_{\chi,i}$
  end if
else
  $m \leftarrow \lfloor (i+j)/2 \rfloor$
  emit $R_\chi[i,j] \leftarrow !R_\chi[i,m]; !R_\chi[m+1,j]$
  $\textsc{Build}(i,m);\ \textsc{Build}(m+1,j)$
end if
end procedure; $\textsc{Build}(1,n)$
emit completion test $c_{q_s,\chi}$: return if monitor $\neq q_s$; otherwise call $!r_{q_s,\chi}$
return $\mathcal{R}_{q_s,\chi}$

A complete transition-repair pass is $\mathrm{Pass}_{q_s,\chi} = R_\chi[1,n] \cdot c_{q_s,\chi}$, where $\cdot$ denotes sequential controller composition. The monitor still advances after every primitive action, but the controller tests source-state exit only after the whole pass. It does not interrupt an atomic call or short-circuit the remaining ranges.

Suppose the monitor leaves q_s while a branch in $\Pi_{\chi,i}$ executes. After that call returns, every suffix leaf either observes its literal without acting or selects only a branch from its already certified $\Pi_{\chi,j}$. Applicability is rechecked at invocation; an applicable call is PDDL-executable and preserves the established signed prefix at module return, while a missing call fails without inserting an uncertified action. Every primitive suffix action is observed by the same DFA, so it may traverse further actual edges, including a rejecting edge, but cannot manufacture a transition or report false acceptance. Only after the suffix finishes does the completion test return to top-level dispatch at the current state or replay when that state is still q_s. Consequently, the controller cannot report an accepting transition that the monitor did not take. The suffix need not be a complete strategy for a newly reached state.

For $n = 1$, $R_\chi[1,1]$ is one leaf followed by the same completion test. Balancing does not choose the certified order or reduce primitive action count; it bounds trigger fanout by two and nesting depth logarithmically while retaining linear work per pass. The repeated state check is an instance of the declarative goal plan pattern (Hübner, Bordini, and Wooldridge 2006); the balanced dispatch structure is GP2PL's controller realization.

5.5 Maintained Domain Library

The atomic compiler constructs $\mathcal{M}_D$ once. Temporal compilation for a query q_i unions its dispatcher and transition plan sets into $\mathcal{Q}_{q_i}$. For an ordered sequence $q_1, \ldots, q_k$, the sole maintained library is

$$\mathcal{L}_D^{[k]} = \mathcal{M}_D \cup \bigcup_{i=1}^{k} \mathcal{Q}_{q_i}, \qquad \mathcal{L}_D^{[0]} = \mathcal{M}_D.$$

The AgentSpeak(L) realization serializes $\mathcal{L}_D^{[k]}$ as one per-domain plan library; $\mathcal{M}_D$ is its stable query-independent subset, not a second library. Appending $\mathcal{Q}_{q_i}$ neither reconstructs nor reoptimizes the atomic core. Query-local goals are control symbols, not PDDL fluents or exported actions. Jason records the complete primitive trace, which VAL checks against the PDDL problem (Howey, Long, and Fox 2004).

If the initial valuation is accepting, the committed plan is empty and the trace is $\langle s_0 \rangle$, not an empty trace or inserted noop; a noop could change strong-Next or Until truth under finite-trace semantics (De Giacomo and Vardi 2013).

6 Conditional Guarantees

The guarantees concern accepted artifacts under their stated premises. They are conditional on the generated certified candidate set and certificate-accepted temporal subset; they do not assert complete candidate generation, universal AgentSpeak optimality, or complete strategy synthesis for arbitrary PDDL domains and DFAs.

Atomic branch soundness. Proposition 1. If an emitted $p(\bar{X})$ branch's context holds under a type-consistent grounding and called modules satisfy their certified achievement and completion summaries, the branch is executable and returns with $p(\bar{X})$. Binding grounds each term; induction over symbolic progression establishes preconditions and final achievement; recursion uses its ranking and preparation rechecks the producer context. □ The proposition establishes local soundness, not reachable-state coverage.

Certified-candidate-set optimality. Proposition 2. For the grounded selection instance defined above, a Clingo optimum $\mathcal{M}_D$ belongs to $\mathfrak{F}_{D,E}$ and minimizes the declared lexicographic cost over that feasible family.

The Technical Supplement proves a bidirectional correspondence between answer sets and feasible subsets. The result does not imply complete candidate generation or optimality over all AgentSpeak programs.

Supported transition-composition soundness. Proposition 3. Let $\mathcal{O}_\chi = \langle P_\chi^+, P_\chi^- \rangle$ be an accepted signed transition obligation. If each required signed repair has an applicable certified branch, each selected call terminates, and later repairs preserve earlier obligations, then at the end of a complete transition-repair pass the controller retries exactly when the monitor reports the source state; otherwise it returns to dispatch at the current state. Every intervening state change

is an actual DFA edge enabled by the observed valuation, and a rejecting state cannot report success.

Proof sketch. Induct over the certified order: each leaf establishes its signed literal; later summaries exclude deleting prior positives or adding forbidden atoms. If the source is left during a call, the remaining suffix still contains only these certified calls and observations, so it preserves the prefix at return boundaries. The monitor evaluates full cubes after every action and cannot be written by the controller; the suffix can therefore cause only real, fully observed DFA transitions. After the full pass, the completion plan tests the current state and either retries or returns to top-level dispatch. □
This does not imply complete action-strategy synthesis or guarantee that a post-exit suffix avoids a rejecting state.

Balanced transition-repair complexity. Proposition 4. For n signed literals and e repair plans, the tree has $2n+e+2$ plans, fan-out two, depth $O(\log n)$, and $O(n)$ visits per pass, while preserving order. The count follows from n satisfied leaves, e repair leaves, $n-1$ internal nodes, one entry, and two completion plans; midpoint splitting and range induction establish depth and order. □

Monitor trace fidelity. Proposition 5. Let $\xi = s_0, a_1, s_1, \ldots, a_k, s_k$ be the PDDL execution observed by the monitor, and let $z_0 = \delta_q(q_q^0, val_q(s_0))$. After every successful primitive action a_i, the runtime monitor state equals $z_i = \delta_q(z_{i-1}, val_q(s_i))$. Hence leaving a controller source state is exactly an exit in the deterministic automaton run over the observed finite trace.

The result follows by induction on primitive actions: the environment first applies the PDDL successor and then evaluates the deterministic transition function on the resulting valuation. Controller plans cannot write monitor beliefs. □

Initial-acceptance trace semantics. Proposition 6. If the initial valuation is accepting, successful execution commits no primitive action and denotes the one-state finite trace $\langle s_0 \rangle$. It is therefore equivalent to evaluating the query on the initial state and is not equivalent to inserting an arbitrary noop.

This is immediate from finite-trace semantics and Proposition 5: the initial valuation is consumed before action execution, while a noop would introduce a second observed position and could change strong-Next or Until truth. □

Complete proofs, including the binding, symbolic-progression, relational-ranking, resource-discharge, and tree-order lemmas used by these propositions, appear in the Technical Supplement, Sec. 2.

7 Experimental Evaluation

Figure 3 summarizes the paired atomic and temporal ablations at domain, stress-family, and execution levels; the variants, validation oracles, and timing boundary are defined below.

7.1 Questions and Comparisons

Atomic compilation. Evidence Only validates provider macros against the PDDL schemas and retains them without producible-target expansion. Direct Producers adds action-only producers for the producible target universe; Maximum Feasible retains a maximum-cardinality jointly feasible subset after adding compatible recursive precondition-repair and target-preserving resource-discharge candidates; and Full GP2PL applies the lexicographic Clingo objective to the same generated certified set. Their paired comparison measures target coverage, internal-call closure, held-out execution, and compact selection of the atomic core.

Temporal composition. Unprotected Serialization uses the same automaton, library, and primitive-step monitor without completion-effect analysis. Certified Flat adds threat-induced precedence ordering and preservation portfolios; Certified Balanced changes only flat control to the balanced binary transition-repair tree. A Module-Return Monitor observes only on module return, testing the primitive-step observation boundary.

End-to-end references. End-to-end evaluation reports valid-trace coverage and keeps one-time compilation, query append, and execution costs separate. Raw MOOSE, LAMA, ENHSP, and FOND4LTLf with LAMA (Chen et al. 2026; Richter and Westphal 2010; Scala et al. 2020; De Giacomo and Fuggitti 2021) are task-level references, not compiler ablations. The MOOSE comparison is partitioned by provenance before scoring. For all twelve domains in its original benchmark, we report the five-model mean coverage from Table 4 of the extended paper (Chen et al. 2025). For all four GP2PL-added domains, we measure Raw MOOSE locally over seeds 0–4. We never select a source separately for an individual domain after observing its outcome.

7.2 Benchmarks and Protocol

Corpus and repetitions. The corpus comprises eight classical and four bounded-integer MOOSE domains with official splits (Chen et al. 2026; Chen 2026), plus four serialized-width families (Lipovetzky and Geffner 2012). Blocksworld Clear and On use published no-constants splits (Francès, Bonet, and Geffner 2021; Bonet 2026); Tower uses the first quarter of a fixed IPC corpus (Bacchus 2001; Potassco 2026), and Depots the first half of the 22-instance D2L corpus (Bonet and Geffner 2018; RLeap Project 2026). Groups are used only for reporting. Following MOOSE (Chen et al. 2026), the atomic study uses all training instances and three goal orders to compile five independently seeded atomic cores. Source revisions, split construction, resource limits, and reporting rules appear in the Technical Supplement, Secs. 4.3 and 5.2–5.3.

Temporal benchmark construction. For each held-out problem, benchmark construction ignores the original achievement goal and derives candidate temporal milestones from legal, state-changing, nonrepeating rollouts of at most three actions; it invokes no planner. Predicate and bounded-integer changes instantiate five predeclared profiles: $F(A \wedge B)$, $F(A \wedge \neg B)$, $F(A \wedge X(F(B)))$, $F(A \wedge X(F(B \wedge X(F(C)))))$, and strong $A\,U\,B$. For example, a witness that first establishes `holding(b1)` and later `on(b1,b4)`

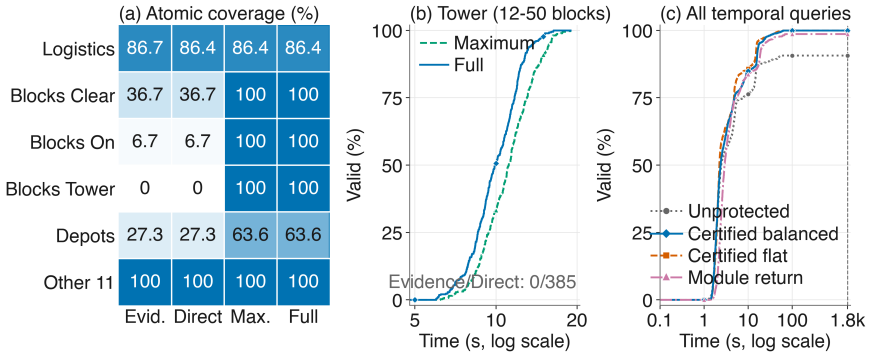

Figure 3: Paired ablations. Panel (a) reports descriptive coverage requiring Jason completion and original-goal VAL acceptance over five evidence seeds for every domain on which the atomic variants differ; the other 11 domains are aggregated because all variants reach 100%. Panel (b) gives cumulative valid traces over end-to-end time for all 385 seed–instance pairs in the registered 12–50-block Tower series. Panel (c) gives cumulative valid-trace coverage for all 1,228 paired temporal queries. Failures remain in every denominator, and the dashed line marks the 1,800-second limit. Repeated atomic seeds are not treated as independent inferential units; numeric labels, markers, and line styles duplicate color.

yields the lifted formula $F(holding(X) \wedge X(F(on(X,Y))))$ under a typed hidden binding.

Each candidate retains a legal witness. Its objects are lifted to typed parameters, and selection balances profile use and alpha-normalized semantic-signature use before applying a symbol-independent tie-break. The resulting benchmark contains one witness-backed query for each of the 1,228 test problems. Requests with identical translation inputs and agreeing hidden semantics are translated once, yielding 475 distinct translation inputs while retaining all 1,228 queries for execution. The complete construction and registered bounds appear in the Technical Supplement, Sec. 4.1.

Controlled inputs and references. Atomic variants share each seed's evidence; temporal variants are paired on the same atomic core, binding, and DFA. MOOSE reuses its learned model; LAMA (Richter and Westphal 2010) and ENHSP MRP+HJ (Scala et al. 2020) are the classical and numeric references used by MOOSE (Chen et al. 2026). FOND4LTLf (De Giacomo and Fuggitti 2021) with LAMA covers its accepted grounded Boolean subset. The published and locally measured MOOSE rows use preregistered disjoint scopes; their source labels and reporting boundaries are given with the external references after the GP2PL results.

Validation and measures. A temporal success requires Jason completion (Bordini, Hübner, and Wooldridge 2007), VAL accepts the full primitive trace against a neutral PDDL goal (Howey, Long, and Fox 2004), and independent replay is accepted by both DFAs. Atomic validation instead uses the original PDDL goal. We report internal-call closure, held-out success, library size, runtime, action count, and trigger fan-out; plan length uses jointly solved instances. Figure 3 follows MOOSE's absolute-time presentation (Chen et al. 2026): coverage is credited only to VAL-valid traces, and all failures remain in the denominator. The generated tables report paired component effects, full-system robustness, temporal profiles, and external scope separately. The Technical Supplement preserves every domain–seed cell, diagnostic, and fixed temporal input. The result subsections first isolate component effects, then evaluate the selected system, and finally place the external references in context.

Atomic totals contain five seeded outcomes for each held-out case identifier. For inference, we average the paired binary difference within each identifier and apply a two-sided exact sign test to the nonzero case-level differences. Temporal queries share domains and, in some cases, one of 475 translated inputs; we therefore report discordant counts and domain concentration without a case-level p-value. Runtime, plan length, and library size remain descriptive.

7.3 Paired Component Ablations

All four atomic variants compile all 80 seed–domain libraries. Evidence Only reaches 90/365 producible predicates, whereas every schema-expanded variant reaches 365/365. Direct Producers differs from Evidence Only on one execution, reflecting AgentSpeak plan choice rather than an infrastructure failure.

The decisive atomic change is certified recursive candidate generation. Maximum Feasible adds 640 valid executions over Direct Producers, an increase of 10.42 percentage points. All 640 seed–case gains occur in the four GP2PL-added serialized-width domains.

After aggregating the five seeded outcomes within each identifier, there are 130 nonzero case-level differences, all favoring Maximum Feasible. The two-sided exact sign test gives $p = 1.47 \times 10^{-39}$, so the conclusion does not rely on treating repeated seed–case outcomes as independent.

Full GP2PL preserves the Maximum Feasible success set while removing 34 mean branches and 10.8 mean KiB, reductions of 2.2% and 1.7%, respectively. This isolates compact selection without assigning it a coverage gain.

The additional capability has an offline cost: Full compiles in 40.7 seconds on average, roughly fivefold the 8.0 seconds of Direct Producers. We report that cost directly and make no unmeasured amortization claim.

Completion-effect ordering and preservation portfolios

Method	Compiled	Targets	Valid	Branches	KiB	Compile s
Evidence Only	80/80	90/365	5420/6140	835.0 ± 15.8	488.3 ± 8.3	5.9 ± 0.7
Direct Producers	80/80	365/365	5419/6140	1153.0 ± 15.8	549.2 ± 8.3	8.0 ± 2.5
Maximum Feasible	80/80	365/365	**6059/6140**	1533.0 ± 15.8	645.9 ± 8.3	41.5 ± 13.1
Full GP2PL	80/80	365/365	**6059/6140**	**1499.0 ± 15.8**	**635.1 ± 8.3**	40.7 ± 7.6

Table 3: Paired atomic compiler comparison over 80 domain libraries and 6,140 seed–case executions: 16 domains and 1,228 held-out cases under five fixed evidence seeds. Compiled counts libraries; Targets uses the shared positive-add-effect predicate denominator; Valid requires Jason success and original-goal VAL. KiB is library size in kibibytes, and Compile s is seconds; Branches, KiB, and Compile s are mean ± sample standard deviation across seeds. Valid totals are descriptive; case-clustered inference is reported in the text. Bold marks tied best coverage; blue bold identifies Full and its lower size within the equal-coverage pair. Color is not used alone.

Method	Built	Valid	PAR-2 s	Actions	Plans	Fan-out
Unprotected Serialization	1228/1228	1113/1228	342.0	2	7	3
Certified Flat	1228/1228	**1227/1228**	8.0	1	7	3
Certified Balanced	1228/1228	**1227/1228**	8.6	1	13	**2**
Module-Return Monitor	1228/1228	1211/1228	56.0	1	13	2

Table 4: Paired temporal compiler comparison over 1,228 queries with identical DFA, binding, and atomic-library inputs. Built counts generated controllers; Valid requires Jason, neutral-goal VAL, and acceptance by the gold and predicted DFA trace oracles. PAR-2 charges failures twice the 1,800-second limit. Actions are medians over the 1097 jointly solved queries; Plans is the median controller size and Fan-out its maximum transition-repair value. Bold marks tied best coverage; blue bold identifies Balanced and its lower fan-out relative to equal-coverage Flat. Color is not used alone.

produce a net gain of 114 valid traces, or 9.28 percentage points, over Unprotected Serialization. Of the discordant queries, 115 favor Certified Flat and 1 favors the unprotected controller. Four bounded-integer domains account for 113 of the 114 net temporal gains, locating the observed benefit in preservation-sensitive numeric composition rather than uniformly across the corpus.

Certified Flat and Certified Balanced have the same 1227-case success set.

Balancing therefore receives no semantic credit. It increases the median controller size from 7 to 13 plans and PAR-2 from 8.0 to 8.6 seconds, while reducing maximum transition-repair fan-out from 3 to 2. We select Balanced for this structural bound, not for coverage or runtime.

Moving observation from primitive steps to module return loses 16 valid traces. Module-Return Monitor records 15 Jason timeouts, 1 Jason failure, and 1 gold-DFA rejection. The shared Certified Flat/Balanced failure is a strategy-choice boundary: the selected branch leaves the monitor in its source state. The Technical Supplement records the case identifier and complete diagnostic.

The paired matrix conditions on its same-run seed-0 Full GP2PL library. Its Certified Balanced row is an ablation estimate; the next subsection evaluates the separately preregistered full-system configuration.

7.4 Full-System Coverage and Robustness

Five-seed atomic robustness. Across five independently compiled atomic cores, mean held-out coverage is 98.68% ± 1.29 percentage points. Fourteen of the sixteen domains are complete. Figure 3 accounts for the entire corpus: it shows each domain with a variant difference and aggregates only the 11 domains on which every variant is complete.

Variation is concentrated in Logistics and Depots. In Logistics, seeded goal order changes the available cross-mode provider macros; the remaining producer dependency is cyclic and lacks a decreasing certificate.

Depots requires a nested support-resource choice beyond the current bound-capacity certificate. These failures delimit candidate construction; every reported success still satisfies original-goal VAL.

No domain-specific rule was introduced after observing held-out outcomes. Table 5 gives the non-pooled summary; the Technical Supplement, Sec. 5.3 gives every domain–seed diagnostic.

Scope	Cases/seed	Valid/seed	Coverage (%)
All 16 domains	1,228	1,187–1,224	98.7 ± 1.3
All-seed complete (14)	1,127	1,127	100.0 ± 0.0
Logistics	90	54–90	86.4 ± 16.7
Depots	11	6–9	63.6 ± 11.1

Table 5: Full GP2PL atomic held-out coverage over $n = 5$ atomic cores compiled independently from predeclared evidence seeds. Coverage is mean ± sample standard deviation (SD) across seeds; Valid/seed gives the minimum–maximum count. Domains complete in every seed are aggregated, and every remaining domain is shown separately. Repeated outcomes are not pooled.

End-to-end temporal validation. The protocol exposes the public PDDL vocabulary, typed parameters, and binding constraints. It admits F, X, strong U, conjunction, and literal

negation.

The five registered structures instantiate Φ_{bench}, rather than exhausting Φ_{syn}. Execution includes only queries admitted to $\Phi_{\mathrm{cert}}(D, \mathcal{M}_D)$.

The controlled source utterances define the evaluation distribution, not a required user grammar.

GPT-5.5 (OpenAI 2026) translates 475 deduplicated specifications. Every accepted translation passes typed validation and exact finite-trace language equivalence, yielding 1,228 bound queries.

Table 6 separates five formula profiles. All 1,228 queries satisfy the end-to-end criterion: Jason completes, all 1,228 nonempty traces pass VAL, and both DFAs accept all 1,228 traces.

Runtime is 5.5 seconds median (interquartile range 4.28–8.49); action count is 2 median (interquartile range 1–2).

Profile	DFA equiv.	E2E valid	Med. act.	Med. s
Ordered-3	137/137	272/272	3	5.4
Ordered-2	136/136	273/273	2	5.6
Strong Until	119/119	275/275	1	5.4
Conjunction	19/19	137/137	1	5.4
Conj.+negation	64/64	271/271	1	5.6
All	475/475	1,228/1,228	2	5.5

Table 6: Translation and execution by temporal profile. DFA equiv. counts the 475 distinct translation inputs whose gold and predicted DFAs have exactly the same finite-trace language. E2E valid counts all 1,228 bound queries and requires controller compilation, Jason completion, neutral-goal VAL, and acceptance by both DFA trace oracles. Medians are primitive actions and wall-clock seconds; the All row is pooled across all bound queries.

Unlike the paired estimate above, this study fixes one preregistered seed-0 atomic core per domain and obtains 1,228/1,228. It supports neither cross-seed claims nor arbitrary PDDL–LTL_f completeness.

The translation protocol appears in the Technical Supplement, Sec. 4.2; Sec. 5.3 gives semantic-conformance cases, distributions, and provenance.

7.5 External Planning References

The published MOOSE row is coverage-only: it is labelled Reported rather than locally reproduced, and its time and memory are not compared across hardware. The local Raw MOOSE extension row is labelled Measured over five models.

The original and added scopes contain 1,080 and 148 cases per seed. Across the four added domains, Raw MOOSE obtains 15.81% $\pm$ 1.56 percentage points.

On the same 740 added-domain seed–case evaluations, Full GP2PL validates 720/740 traces, whereas Raw MOOSE validates 117/740. This descriptive same-scope contrast isolates the practical value of compiling evidence into a closed recursive library on the serialized-width families.

Because these scopes are disjoint, their difference is not a same-instance comparison. Moreover, published Raw MOOSE coverage on its original 12-domain scope is already near ceiling. We therefore claim improved structural coverage on the added families, not universal dominance wherever Raw MOOSE already saturates.

Table 7 records the preregistered source split and coverage boundary.

Scope	Valid/seed	Coverage (%)
Reported (MOOSE Table 4)		
Original 12 domains	1079.6/1,080	99.96
Measured (local five seeds)		
GP2PL-added 4	23.4 $\pm$ 2.3/148	15.81 $\pm$ 1.56
Blocks Clear	7.0 $\pm$ 0.0/30	23.33 $\pm$ 0.00
Blocks On	2.0 $\pm$ 0.0/30	6.67 $\pm$ 0.00
Blocks Tower	6.0 $\pm$ 1.7/77	7.79 $\pm$ 2.25
Depots	8.4 $\pm$ 1.1/11	76.36 $\pm$ 10.37

Table 7: Raw MOOSE coverage under the preregistered source split. The Reported row is copied from Table 4 of the five-seed MOOSE extended paper (Chen et al. 2025); the Measured rows are mean $\pm$ sample standard deviation over the five local evidence seeds. The scopes are disjoint, and no cross-hardware runtime comparison is made.

Instance-planner outcomes. LAMA validates 591/868 classical instances, while MRP+HJ validates 253/360 numeric instances. Their PAR-2 values are 1178.4 and 1127.4 seconds.

These rows cover disjoint fragments and are interpreted within scope rather than as a same-instance ranking. FOND4LTLf with LAMA supports 492 of 1228 temporal queries and validates 298.

The remaining 736 inputs are unsupported rather than planner failures; this total includes both numeric PDDL outside the tool's scope and identifiers rejected by the current adapter encoding. Supported-set PAR-2 is 1457.5 seconds. Raw MOOSE extension runtime remains omitted under its coverage-only protocol. The Technical Supplement reports the planner, compiler, timeout, and unsupported outcome taxonomy.

Method	Source	Scope	Coverage	Unsupported	PAR-2 s
MOOSE	Reported	Original MOOSE domains, five seeds	1079.6/1080	0	–
Raw MOOSE extension	Measured	Added domains, five seeds	23.4 ± 2.3/148	0	–
LAMA	Measured	Classical achievement	591/868	0	1178.4
MRP+HJ	Measured	Numeric achievement	253/360	0	1127.4
FOND4LTLf + LAMA	Measured	Supported Boolean TEG	298/492	736	1457.5

Table 8: Scope-separated external planning references. Reported MOOSE coverage is copied from Table 4 of the five-seed extended paper (Chen et al. 2025); its runtime is not compared with local measurements. Measured rows use a 1,800-second, 8-GiB per-task budget, and PAR-2 charges nonvalid supported cases twice the cutoff. Raw MOOSE extension coverage is mean $\pm$ sample standard deviation over five seeds; its runtime is omitted by protocol. FOND4LTLf unsupported inputs are excluded from its coverage denominator and separated from planner and compiler failures. Rows with different scopes are not ranked.

8 Conclusion and Future Work

GP2PL closes the representation gap by converting generalized-planning evidence and PDDL action schemas into a reusable atomic BDI core, then appending query-local temporal controllers to the same maintained library.

The output is executable in the evaluated AgentSpeak(L)/Jason realization because each accepted branch has certified binding, symbolic PDDL executability, target achievement, and progress, while selected internal calls resolve within the library. Unproved obligations fail compilation rather than becoming unchecked plans.

The certified temporal scope consists of bound formulas in the declared F, X, strong-U, conjunction, and literal-negation syntax whose DFA progress obligations admit preservation-safe repairs. Within that scope, the measured component gains and 1,228/1,228 fixed-core temporal result show that certification changes both coverage and controller structure, while the conditions below delimit what those results establish.

MOOSE is the only instantiated evidence provider; the candidate-construction grammar remains incomplete beyond the implemented acyclic, ranking, and keyed-resource certificates, and call closure proves type-compatible resolution rather than universal applicability.

The temporal study uses controlled, witness-backed queries mined from at most three actions and a fixed seed-0 atomic core, so it does not measure unrestricted language, long-horizon planning, or cross-seed temporal robustness.

Primitive-step monitoring is exact for the declared syntax, but rejected arithmetic, interference, and repair cases preclude arbitrary PDDL–LTL$_f$ strategy synthesis.

Future work can use a parameterized PDDL problem generator to broaden evidence acquisition through a pinned benchmark source, as in planning competitions (Bacchus 2001).

After reserving a disjoint held-out split and validating every generated problem against D, a compatible generalized-planning backend can learn evidence from $\mathcal{I}_{\mathrm{train}}$ for the unchanged GP2PL compiler. This scales evidence acquisition and domain onboarding, not the supported PDDL–LTL$_f$ fragment or representative-coverage guarantees.

References

Alcázar, V.; Borrajo, D.; Fernández, S.; and Fuentetaja, R. 2013. Revisiting Regression in Planning. In *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence*, 2254–2260. AAAI Press.

Bacchus, F. 2001. The AIPS '00 Planning Competition. *AI Magazine*, 22(3): 47–56.

Beurer-Kellner, L.; Fischer, M.; and Vechev, M. T. 2023. Prompting Is Programming: A Query Language for Large Language Models. *Proceedings of the ACM on Programming Languages*, 7(PLDI).

Bonassi, L.; De Giacomo, G.; Favorito, M.; Fuggitti, F.; Gerevini, A. E.; and Scala, E. 2023. Planning for Temporally Extended Goals in Pure-Past Linear Temporal Logic. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 33, 61–69. Palo Alto, California: AAAI Press.

Bonet, B. 2026. Learner Policies from Examples: Benchmark Repository. GitHub repository. Revision 9991926f7655c4b6c8dc2f0404123639e42056f2.

Bonet, B.; Drexler, D.; and Geffner, H. 2024. On Policy Reuse: An Expressive Language for Representing and Executing General Policies that Call Other Policies. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 34, 31–39. Palo Alto, California: AAAI Press.

Bonet, B.; and Geffner, H. 2018. Features, Projections, and Representation Change for Generalized Planning. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence*, 4667–4673. Stockholm, Sweden: International Joint Conferences on Artificial Intelligence Organization.

Bonet, B.; and Geffner, H. 2024. General Policies, Subgoal Structure, and Planning Width. *Journal of Artificial Intelligence Research*, 80: 475–516.

Bordini, R. H.; Hübner, J. F.; and Wooldridge, M. 2007. *Programming Multi-Agent Systems in AgentSpeak Using Jason*. John Wiley & Sons.

Chen, D. Z. 2026. MOOSE Companion Benchmark Dataset. GitHub repository. Revision e00970516154e9042b783a4613a1ed7286c9beee.

Chen, D. Z.; Hofmann, T.; Klassen, T. Q.; and McIlraith, S. A. 2025. Satisficing and Optimal Generalised Planning via Goal Regression. Version 1, arXiv:2511.11095.

Chen, D. Z.; Hofmann, T.; Klassen, T. Q.; and McIlraith, S. A. 2026. Satisficing and Optimal Generalised Planning via Goal Regression. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 36197–36206. Palo Alto, California: AAAI Press.

Chen, Y.; Gandhi, R.; Zhang, Y.; and Fan, C. 2023. NL2TL: Transforming Natural Languages to Temporal Logics Using Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 15880–15903. Singapore: Association for Computational Linguistics.

De Giacomo, G.; and Fuggitti, F. 2021. FOND4LTLf: FOND Planning for LTLf/PLTLf Goals as a Service. In *Proceedings of the International Conference on Automated Planning and Scheduling: System Demonstrations*.

De Giacomo, G.; and Rubin, S. 2018. Automata-Theoretic Foundations of FOND Planning for LTLf and LDLf Goals. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence*, 4729–4735. International Joint Conferences on Artificial Intelligence Organization.

De Giacomo, G.; and Vardi, M. Y. 2013. Linear Temporal Logic and Linear Dynamic Logic on Finite Traces. In *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence*, 854–860. AAAI Press.

De Silva, L.; Meneguzzi, F.; and Logan, B. 2020. BDI Agent Architectures: A Survey. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*, 4914–4921. International Joint Conferences on Artificial Intelligence Organization.

Drexler, D.; Seipp, J.; and Geffner, H. 2022. Learning Sketches for Decomposing Planning Problems into Subproblems of Bounded Width. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 32, 62–70. Palo Alto, California: AAAI Press.

Drexler, D.; Seipp, J.; and Geffner, H. 2024. Expressing and Exploiting Subgoal Structure in Classical Planning Using Sketches. *Journal of Artificial Intelligence Research*, 80: 171–208.

Fikes, R. E.; Hart, P. E.; and Nilsson, N. J. 1972. Learning and Executing Generalized Robot Plans. *Artificial Intelligence*, 3: 251–288.

Fikes, R. E.; and Nilsson, N. J. 1971. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving. *Artificial Intelligence*, 2(3–4): 189–208.

Fox, M.; and Long, D. 2003. PDDL2.1: An Extension to PDDL for Expressing Temporal Planning Domains. *Journal of Artificial Intelligence Research*, 20: 61–124.

Francès, G.; Bonet, B.; and Geffner, H. 2021. Learning General Planning Policies from Small Examples Without Supervision. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 11801–11808. Palo Alto, California: AAAI Press.

Fuggitti, F. 2020. LTLf2DFA: Translate LTLf and PLTLf Formulae to Deterministic Finite Automata.

Fuggitti, F.; and Chakraborti, T. 2023. NL2LTL—A Python Package for Converting Natural Language Instructions to Linear Temporal Logic Formulas. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, 16428–16430. AAAI Press.

Gebser, M.; Kaminski, R.; Kaufmann, B.; and Schaub, T. 2019. Multi-shot ASP Solving with Clingo. *Theory and Practice of Logic Programming*, 19(1): 27–82.

Guo, W.; Kingston, Z.; and Kavraki, L. E. 2025. CaStL: Constraints as Specifications Through LLM Translation for Long-Horizon Task and Motion Planning. In *2025 IEEE International Conference on Robotics and Automation*, 11957–11964. IEEE.

Henriksen, J. G.; Jensen, O. J. L.; Jørgensen, M. E.; Klarlund, N.; Paige, R.; Rauhe, T.; and Sandholm, A. B. 1995. MONA: Monadic Second-Order Logic in Practice. *BRICS Report Series*, 2(21).

Hindriks, K. V.; van der Hoek, W.; and van Riemsdijk, M. B. 2009. Agent Programming with Temporally Extended Goals. In *Proceedings of the 8th International Conference on Autonomous Agents and Multiagent Systems*, 137–144. International Foundation for Autonomous Agents and Multiagent Systems.

Howey, R.; Long, D.; and Fox, M. 2004. VAL: Automatic Plan Validation, Continuous Effects and Mixed Initiative Planning Using PDDL. In *Proceedings of the 16th IEEE International Conference on Tools with Artificial Intelligence*, 294–301. IEEE Computer Society.

Hübner, J. F.; Bordini, R. H.; and Wooldridge, M. 2006. Programming Declarative Goals Using Plan Patterns. In *Declarative Agent Languages and Technologies IV*, volume 4327 of *Lecture Notes in Computer Science*, 123–140. Springer.

Jiménez, S.; Segovia-Aguas, J.; and Jonsson, A. 2019. A Review of Generalized Planning. *The Knowledge Engineering Review*, 34: e5.

Lipovetzky, N.; and Geffner, H. 2012. Width and Serialization of Classical Planning Problems. In *Proceedings of the 20th European Conference on Artificial Intelligence*, volume 242 of *Frontiers in Artificial Intelligence and Applications*, 540–545. IOS Press.

Liu, J. X.; Yang, Z.; Idrees, I.; Liang, S.; Schornstein, B.; Tellex, S.; and Shah, A. 2023. Grounding Complex Natural Language Commands for Temporal Tasks in Unseen Environments. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, 1084–1110. PMLR.

McDermott, D.; Ghallab, M.; Howe, A.; Knoblock, C.; Ram, A.; Veloso, M.; Weld, D.; and Wilkins, D. 1998. PDDL—The Planning Domain Definition Language.

Meneguzzi, F.; and De Silva, L. 2015. Planning in BDI Agents: A Survey of the Integration of Planning Algorithms and Agent Reasoning. *The Knowledge Engineering Review*, 30(1): 1–44.

Meneguzzi, F.; Fraga Pereira, R.; and Oren, N. 2025. Generalised BDI Planning. In *Proceedings of the 24th International Conference on Autonomous Agents and Multia-*

gent Systems, 1483–1491. International Foundation for Autonomous Agents and Multiagent Systems.

Necula, G. C. 1997. Proof-Carrying Code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 106–119. ACM.

OpenAI. 2026. GPT-5.5 System Card. OpenAI system card.

Potassco. 2026. PDDL Instances from the International Planning Competitions. GitHub repository. Revision cf19edf7c53d1540ddbb396c642595e0926ee552.

Rao, A. S. 1996. AgentSpeak(L): BDI Agents Speak Out in a Logical Computable Language. In *Agents Breaking Away*, Lecture Notes in Computer Science, 42–55. Berlin, Heidelberg: Springer Berlin Heidelberg.

Richter, S.; and Westphal, M. 2010. The LAMA Planner: Guiding Cost-Based Anytime Planning with Landmarks. *Journal of Artificial Intelligence Research*, 39: 127–177.

RLeap Project. 2026. D2L: Description Logic Features for Planning. GitHub repository. Revision 0620e169c894d79b3c84f435dba1462996f7c270.

Sardina, S.; and Padgham, L. 2007. Goals in the Context of BDI Plan Failure and Planning. In *Proceedings of the 6th International Joint Conference on Autonomous Agents and Multiagent Systems*, 1–8. ACM.

Scala, E.; Saetti, A.; Serina, I.; and Gerevini, A. E. 2020. Search-Guidance Mechanisms for Numeric Planning Through Subgoaling Relaxation. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 30, 226–234. AAAI Press.

Segovia-Aguas, J.; E-Martín, Y.; and Jiménez, S. 2022. Representation and Synthesis of C++ Programs for Generalized Planning. *CoRR*, abs/2206.14480.

Srivastava, S.; Immerman, N.; Zilberstein, S.; and Zhang, T. 2011. Directed Search for Generalized Plans Using Classical Planners. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 21, 226–233. AAAI Press.

Thangarajah, J.; Padgham, L.; and Winikoff, M. 2003. Detecting and Avoiding Interference Between Goals in Intelligent Agents. In *Proceedings of the Eighteenth International Joint Conference on Artificial Intelligence*, 721–726. Morgan Kaufmann.

Xu, M.; Lumley, T.; Fraga Pereira, R.; and Meneguzzi, F. 2024. A Practical Operational Semantics for Classical Planning in BDI Agents. In *ECAI 2024—27th European Conference on Artificial Intelligence*, volume 392 of *Frontiers in Artificial Intelligence and Applications*, 1365–1372. IOS Press.

Yang, R.; Silver, T.; Curtis, A.; Lozano-Pérez, T.; and Kaelbling, L. P. 2022. PG3: Policy-Guided Planning for Generalized Policy Generation. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*, 4686–4692. Vienna, Austria: International Joint Conferences on Artificial Intelligence Organization.

Zhuo, T. Y.; Li, Z.; Huang, Y.; Shiri, F.; Wang, W.; Haffari, G.; and Li, Y.-F. 2023. On Robustness of Prompt-Based Semantic Parsing with Large Pre-Trained Language Model: An Empirical Study on Codex. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, 1090–1102. Dubrovnik, Croatia: Association for Computational Linguistics.

Reproducibility Checklist

Instructions for Authors:

This document outlines key aspects for assessing reproducibility. Please provide your input by editing this `.tex` file directly.

For each question (that applies), replace the "Type your response here" text with your answer.

Example: If a question appears as

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
Type your response here
```

you would change it to:

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
yes
```

Please make sure to:

- Replace ONLY the "Type your response here" text and nothing else.
- Use one of the options listed for that question (e.g., **yes**, **no**, **partial**, or **NA**).
- **Not** modify any other part of the `\question` command or any other lines in this document.

You can `\input` this .tex file right before `\end{document}` of your main file or compile it as a stand-alone document. Check the instructions on your conference's website to see if you will be asked to provide this checklist with your paper or separately.

---

1. General Paper Structure

1.1. Includes a conceptual outline and/or pseudocode description of AI methods introduced (yes/partial/no/NA) yes

1.2. Clearly delineates statements that are opinions, hypothesis, and speculation from objective facts and results (yes/no) yes

1.3. Provides well-marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper (yes/no) yes

2. Theoretical Contributions

2.1. Does this paper make theoretical contributions? (yes/no) yes

If yes, please address the following points:

2.2. All assumptions and restrictions are stated clearly and formally (yes/partial/no) yes

2.3. All novel claims are stated formally (e.g., in theorem statements) (yes/partial/no) yes

2.4. Proofs of all novel claims are included (yes/partial/no) yes

2.5. Proof sketches or intuitions are given for complex and/or novel results (yes/partial/no) yes

2.6. Appropriate citations to theoretical tools used are given (yes/partial/no) yes

2.7. All theoretical claims are demonstrated empirically to hold (yes/partial/no/NA) yes

2.8. All experimental code used to eliminate or disprove claims is included (yes/no/NA) yes

3. Dataset Usage

3.1. Does this paper rely on one or more datasets? (yes/no) yes

If yes, please address the following points:

3.2. A motivation is given for why the experiments are conducted on the selected datasets (yes/partial/no/NA) yes

3.3. All novel datasets introduced in this paper are included in a data appendix (yes/partial/no/NA) yes

3.4. All novel datasets introduced in this paper will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no/NA) yes

3.5. All datasets drawn from the existing literature (potentially including authors' own previously published work) are accompanied by appropriate citations (yes/no/NA) yes

3.6. All datasets drawn from the existing literature (potentially including authors' own previously published work) are publicly available (yes/partial/no/NA) yes

3.7. All datasets that are not publicly available are described in detail, with explanation why publicly available alternatives are not scientifically satisficing (yes/partial/no/NA) yes

4. Computational Experiments

4.1. Does this paper include computational experiments? (yes/no) yes

If yes, please address the following points:

4.2. This paper states the number and range of values tried per (hyper-) parameter during development of the paper, along with the criterion used for selecting the final parameter setting (yes/partial/no/NA) yes

4.3. Any code required for pre-processing data is included in the appendix (yes/partial/no) yes

4.4. All source code required for conducting and analyzing the experiments is included in a code appendix (yes/partial/no) yes

4.5. All source code required for conducting and analyzing the experiments will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no) yes

4.6. All source code implementing new methods have comments detailing the implementation, with references to the paper where each step comes from (yes/partial/no) yes

4.7. If an algorithm depends on randomness, then the method used for setting seeds is described in a way sufficient to allow replication of results (yes/partial/no/NA) yes

4.8. This paper specifies the computing infrastructure used for running experiments (hardware and software), including GPU/CPU models; amount of memory; operating system; names and versions of relevant software libraries and frameworks (yes/partial/no) yes

4.9. This paper formally describes evaluation metrics used and explains the motivation for choosing these metrics (yes/partial/no) yes

4.10. This paper states the number of algorithm runs used to compute each reported result (yes/no) yes

4.11. Analysis of experiments goes beyond single-dimensional summaries of performance (e.g., average; median) to include measures of variation, confidence, or other distributional information (yes/no) yes

4.12. The significance of any improvement or decrease in performance is judged using appropriate statistical tests (e.g., Wilcoxon signed-rank) (yes/partial/no) yes

4.13. This paper lists all final (hyper-)parameters used for each model/algorithm in the paper's experiments (yes/partial/no/NA) yes